Formal Problem Statements

Assignment 3, Parts A and B

14 July 2026

Contents

1 Formal Problem Statements — Assignment 3

> **New to this, or the notation is hard going?** Read `docs/MATH_EXPLAINED.md` alongside this file. It decodes every equation below symbol by symbol, in plain English, and runs each formula with real numbers from our own hall so you can see it work. This file is deliberately written the compressed way a report expects; that one is written the way you actually learn it.

The assignment has two parts, and we chose two separate problems rather than forcing one problem to serve both. They share a setting: a University of Dhaka residential hall. Part A places its Wi-Fi. Part B runs its water pump.

- **Part A** — population-based search. A continuous, non-differentiable, constrained max-coverage problem, solved with a particle swarm optimiser.
- **Part B** — decision making under uncertainty. A finite discounted Markov decision process, solved exactly with Value Iteration and approximately with Q-learning.

---

2 Part A — Wi-Fi Access-Point Placement

2.1 Maximum Weighted Coverage on a University Residential-Hall Floor

2.1.1 1. Informal description

A University of Dhaka residential hall has a fixed, tight budget of K Wi-Fi access points (APs) to mount on one floor. Rooms far from an AP, or shadowed by concrete partitions, receive unusable signal. We must choose the **continuous positions of the K APs on the floor plan** so as to **maximise the occupancy-weighted fraction of rooms that receive a usable link**, subject to a co-channel-interference separation rule and the physical boundary of the floor.

This is a **continuous, constrained, single-objective max-coverage facility-location problem**.

2.1.2 2. Given data (problem instance)

Symbol	Meaning
W, H	floor width and height (metres); floor domain $\Omega = [0, W] \times [0, H]$
N	number of rooms (demand points)
$\mathbf{r}_i = (r_i^x, r_i^y) \in \Omega$	position of room i, $i = 1, \dots, N$
$w_i > 0$	occupancy weight of room i (students)
$\mathcal{W}$	set of interior wall segments (line segments in Ω)
P_{tx}	AP transmit power (dBm)
L_0	reference path loss at $d_0 = 1$ m (dB)
n	log-distance path-loss exponent
γ	attenuation per wall crossing (dB)
τ	usable-link RSSI threshold (dBm)
s	logistic softness around τ (dB)
K	number of APs to place (budget)
$d_{\min}$	minimum allowed separation between two APs (m)
λ_{sep}	separation-penalty weight

2.1.3 3. Decision variables

The positions of the K APs, stacked into one vector:

$$\mathbf{x} = (\mathbf{a}_1, \dots, \mathbf{a}_K) = (a_1^x, a_1^y, \dots, a_K^x, a_K^y) \in \mathbb{R}^{2K}, \qquad \mathbf{a}_j = (a_j^x, a_j^y) \in \Omega.$$

The search space is the box $\mathcal{S} = \Omega^K \subset \mathbb{R}^{2K}$.

2.1.4 4. Signal and coverage model

Distance between room i and AP j (floored at $d_0 = 1$ m, the model's validity radius):

$$d_{ij}(\mathbf{x}) = \max\left(\|\mathbf{r}_i - \mathbf{a}_j\|_2,\ 1\right).$$

Received signal strength (log-distance indoor path loss with discrete wall attenuation):

$$\mathrm{RSSI}_{ij}(\mathbf{x}) = P_{\text{tx}} - \left(L_0 + 10\, n \log_{10} d_{ij}(\mathbf{x})\right) - \gamma\, c_{ij}(\mathbf{x}),$$

where $c_{ij}(\mathbf{x}) \in \mathbb{Z}_{\geq 0}$ is the number of wall segments in $\mathcal{W}$ intersected by the segment $\overline{\mathbf{r}_i\, \mathbf{a}_j}$.

Best-AP soft coverage of room i (each room is served by its strongest AP; a logistic gives a smooth threshold rewarding signal *margin*):

$$\rho_i(\mathbf{x}) = \sigma\left(\frac{\max_{j=1,\dots,K} \mathrm{RSSI}_{ij}(\mathbf{x}) - \tau}{s}\right) \in (0, 1), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Weighted coverage (the quantity to maximise, as a percentage):

$$C(\mathbf{x}) = 100 \cdot \frac{\sum_{i=1}^N w_i\, \rho_i(\mathbf{x})}{\sum_{i=1}^N w_i} \in [0, 100].$$

2.1.5 5. Optimization problem

$$\boxed{\max_{\mathbf{x} \in \mathbb{R}^{2K}} \quad F(\mathbf{x}) = C(\mathbf{x}) \ - \ \lambda_{\text{sep}} \sum_{1 \leq j < k \leq K} \left[\max\left(0,\ d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|_2\right)\right]^2}$$

$$\text{subject to} \qquad 0 \leq a_j^x \leq W, \quad 0 \leq a_j^y \leq H, \qquad j = 1, \dots, K.$$

- The **box constraints** (AP inside the floor) are enforced by **repair** (clamping) during optimization.
- The **separation constraint** ($\|\mathbf{a}_j - \mathbf{a}_k\| \geq d_{\min}$, co-channel interference) is a soft **quadratic penalty** — the second term of F — which is 0 when feasible and grows with the breach.
- The **cardinality constraint** (exactly K APs) is structural: it is baked into the dimension $2K$ of $\mathbf{x}$ and can never be violated.

2.1.6 6. Why the problem is hard (landscape characterisation)

F is:

- **non-differentiable** — it contains $\max_j(\cdot)$ (a kink), a logistic threshold, and the **integer wall-crossing counts** c_{ij}, which make F piecewise-constant in $\mathbf{x}$ (so $\nabla F = 0$ a.e. and undefined on the kinks);
- **non-convex and multimodal** — many spatially distinct layouts yield similar coverage (which AP serves which region is interchangeable), producing many local optima and broad symmetric basins;
- **black-box in structure** — cheap to evaluate pointwise but with no closed-form inverse; only sampling is available.

Hence gradient-based methods have no usable gradient, and exact solvers require discretising Ω — an NP-hard max-coverage selection that explodes combinatorially and discards the continuous optimum. A **population-based metaheuristic (Particle Swarm Optimization)** is the appropriate solver: each particle is a full candidate layout in $\mathbb{R}^{2K}$, and the swarm searches the floor collectively.

2.1.7 7. Reference instance (as implemented)

$$
\begin{aligned}
&W = 60,\ H = 40 \text{ m}; \quad N = 40 \text{ rooms on a jittered } 10 \times 4 \text{ grid}, w_i \in \{1, 2, 3, 4\};\\
&|\mathcal{W}| = 3 \text{ concrete partitions}; \quad K = 3 \ \Rightarrow\ \mathcal{S} = \mathbb{R}^6;\\
&P_{\text{tx}} = 20 \text{ dBm},\ L_0 = 40 \text{ dB},\ n = 3.3,\ \gamma = 8 \text{ dB},\ \tau = -66 \text{ dBm},\ s = 4 \text{ dB};\\
&d_{\min} = 10 \text{ m},\ \lambda_{\text{sep}} = 0.30.
\end{aligned}
$$

Solution quality metric. Alongside the soft objective F, we report **hard coverage** — the unweighted percentage of rooms whose best-AP RSSI meets the threshold, $\frac{100}{N} \sum_i \mathbb{1}[\max_j \text{RSSI}_{ij} \geq \tau]$ — as the deployment-facing measure of how many rooms are actually connected.

---

3 Part B — Pump Control under Load-Shedding

3.1 Minimum-Cost Water Supply for a Residential Hall, as a Markov Decision Process

3.1.1 1. Informal description

The same hall keeps its water in an overhead tank, refilled by an electric pump from an underground reservoir. Three things turn this from plumbing into a decision problem:

1. **The grid fails**, and it fails hardest in the evening, exactly when the hall wants water. Evening outages are also long: once the power goes it tends to stay gone.
2. **A diesel generator** can drive the pump through an outage, but the hall buys a **fixed ration of diesel each morning**. Burn it at 7am on a hunch and there is none left at 9pm.
3. **Demand is not flat.** It spikes when the hall bathes, morning and evening.

Every hour the caretaker picks one action. Pumping early on cheap grid power banks water against an outage that has not happened yet; pumping diesel early spends a ration that cannot be recovered. This is a **finite, discounted, continuing Markov decision process**.

3.1.2 2. Given data (problem instance)

Symbol	Meaning	Value used
$L_{\max}$	tank capacity, in units of 100 L	10
q	pump throughput per hour (units)	3
$D_{\max}$	demand truncation	6
$F_{\max}$	diesel ration delivered each morning (generator-hours)	2
c_{grid}	cost of one grid pump-hour	1.0
c_{gen}	cost of one diesel pump-hour	6.0
c_{short}	cost per unit of water demanded and not delivered	20.0
c_{spill}	cost per unit overflowed	0.5
λ_t	mean hourly demand (truncated Poisson)	peaks 2.5 at 06–09 and 18–22
p_t^{fail}	$P(\text{grid drops} \mid \text{grid up})$	0.02 night / 0.10 day / **0.35 evening**
p_t^{rest}	$P(\text{grid returns} \mid \text{grid down})$	0.50 night / 0.35 day / **0.15 evening**
γ	discount factor	0.99

The evening figures are the heart of the instance: failure is likely *and* restoration is slow, so an evening outage lasts $1/0.15 \approx 6.7$ hours in expectation.

3.1.3 3. State, actions, and availability

$$s = (L,\ t,\ g,\ F) \in \mathcal{S}, \qquad L \in \{0, \dots, L_{\max}\},\ \ t \in \{0, \dots, 23\},\ \ g \in \{0, 1\},\ \ F \in \{0, \dots, F_{\max}\}$$

so $|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = 1584$. The hour wraps, which makes this a **continuing** task, not an episodic one.

$$\mathcal{A} = \{\mathsf{idle},\ \mathsf{pump}_{\text{grid}},\ \mathsf{pump}_{\text{diesel}}\}$$

Actions are **state-dependent**:

$$\mathcal{A}(s) = \{\mathsf{idle}\}\ \cup\ \{\mathsf{pump}_{\text{grid}} : g = 1\}\ \cup\ \{\mathsf{pump}_{\text{diesel}} : F > 0\}$$

Pumping on grid during an outage is not an action, it is a dead switch; offering it would make it an exact duplicate of `idle` in all **792** outage states ($11 \times 24 \times 3$), which wastes a third of the exploration budget on a provable no-op and reduces any policy-agreement metric to a coin flip. Masking leaves $4752 - 792 - 528 = 3432$ legal (s, a) pairs. It cannot change V^* — a duplicate action cannot change a max — so it costs nothing in optimality and buys sample efficiency and honest metrics.

3.1.4 4. Dynamics

Let the inflow and pump cost be

$$q(s,a) = \begin{cases} q & a = \mathsf{pump}_{\text{grid}},\ g = 1 \\ q & a = \mathsf{pump}_{\text{diesel}},\ F > 0 \\ 0 & \text{otherwise} \end{cases} \qquad \kappa(s,a) = \begin{cases} c_{\text{grid}} & a = \mathsf{pump}_{\text{grid}},\ g = 1 \\ c_{\text{gen}} & a = \mathsf{pump}_{\text{diesel}},\ F > 0 \\ 0 & \text{otherwise.} \end{cases}$$

The pump acts, then the students draw. Write $\tilde{L} = \min(L + q(s,a),\ L_{\text{max}})$ for the post-pump level and $O = \max(0,\ L + q(s,a) - L_{\text{max}})$ for the spill. With demand $D \sim \text{Poisson}(\lambda_t)$ truncated to $\{0, \dots, D_{\text{max}}\}$ and **renormalised** (not clipped), the unmet demand and next level are

$$U = \max(0,\ D - \tilde{L}), \qquad L' = \max(0,\ \tilde{L} - D).$$

The grid follows a two-state Markov chain $g' \sim P(\cdot \mid g, t)$, independent of D. The diesel decrements on use and is **redelivered each midnight**:

$$F' = \begin{cases} F_{\text{max}} & t = 23 \quad \text{(the morning drum arrives)} \\ F - \mathbb{1}[a = \mathsf{pump}_{\text{diesel}}] & \text{otherwise,} \end{cases} \qquad t' = (t+1) \bmod 24.$$

3.1.5 5. Reward, and a subtlety that matters

$$r(s,a,w) = -\Big(\kappa(s,a)\ +\ c_{\text{spill}}\, O\ +\ c_{\text{short}}\, U\Big), \qquad w = (D,\ g').$$

The realised shortage U depends on the realised demand D, and D **is not recoverable from** s' — once the tank hits zero, $L' = 0$ tells you nothing about how much demand went unmet. So r is **not** a function of (s, a, s').

This is not a defect. It is the standard *disturbance* form of an MDP. Value Iteration only ever needs the **expected** reward,

$$R(s,a) = \mathbb{E}_w\big[r(s,a,w)\big] = -\Big(\kappa(s,a) + c_{\text{spill}} O + c_{\text{short}} \textstyle\sum_d \Pr[D = d] \max(0, d - \tilde{L})\Big),$$

which we tabulate, and the Bellman operator remains a γ-contraction. Q-learning consumes the **realised** r. It must: feeding it $\mathbb{E}[U]$ would leak the demand distribution into a supposedly model-free agent and collapse the entire distinction the assignment exists to study.

3.1.6 6. The optimization problem

Find a stationary policy $\pi : \mathcal{S} \to \mathcal{A}$ with $\pi(s) \in \mathcal{A}(s)$ maximising

$$\boxed{V^\pi(s) = \mathbb{E}\Big[\textstyle\sum_{k=0}^{\infty} \gamma^k\, r_{t+k}\ \Big|\ s_t = s,\ \pi\Big]}$$

The optimum satisfies the Bellman optimality equation

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \Big[R(s,a) + \gamma \textstyle\sum_{s'} P(s' \mid s, a)\, V^*(s')\Big].$$

Choice of criterion. We use discounted return, not average reward. At one hour per step, $\gamma = 0.99$ gives an effective horizon $1/(1-\gamma) = 100$ hours, comfortably longer than the 24-hour cycle the problem turns on. For a continuing operations problem average cost per hour is arguably more natural, and the discounted-optimal policy is not guaranteed to be gain-optimal; we report discounted return consistently everywhere and never mix the two axes.

3.1.7 7. Why both algorithms belong here

$P(s' \mid s, a)$ can be written down, so **Value Iteration** applies and returns the exact optimum. That is what makes it *model-based*: the sum over s' is only computable because somebody handed us P.

Q-learning is given only a simulator it can poke — one (s', r) at a time, no probabilities. That is what *model-free* means.

The scientific question is therefore not "which one wins" (with a correct model VI wins by construction, it is exact). It is: **how wrong must the model be, and how much experience must you buy, before learning beats planning?** Dhaka publishes a load-shedding schedule that is famously optimistic, so this is not hypothetical.

We also run the arm that can falsify the obvious thesis: estimate $\hat{P}$ from Q-learning's *own* samples and plan on that (certainty equivalence). If a learned model beats the learner on identical data, then "model-free wins when the model is wrong" is the wrong lesson.

3.1.8 8. Reporting metrics

Every policy — Value Iteration's, Q-learning's, certainty-equivalence's, and the baselines' — is scored the same way: by **exactly solving** $(I - \gamma P_\pi)V^\pi = R_\pi$ under the **true** model. This removes Monte-Carlo noise from the comparison completely. Policies are ranked by exact expected return, not by which one drew a luckier rollout.

We report **regret**, $100 \cdot (V^* - V^\pi)/|V^*|$, plus rollout statistics (cost per day, shortage units per day, diesel hours per day) purely to translate abstract return into consequences a hall warden would recognise.

Companion documents: alternatives considered in `docs/PART4_problem_design.md`; viva preparation in `docs/PART3_viva.md`; solvers, baselines and results in `src/pso_wifi_placement.py`, `src/rl_water_tank.py`, `src/rl_experiments.py`, and the README.